Supplemental Material

More Evidence for Low-Rank Structure of Join Graph Matrices

To further characterize the low-rank structure of join graph matrices, we compute two additional measures derived from the singular value decomposition (SVD) as suggested by Reviewer 3. The SVD decomposes a matrix into singular values $\sigma_1 \geq \sigma_2 \geq \cdots \geq \sigma_r$, which represent the "energy" distributed across the rank-r subspace. This energy quantifies the contribution of each singular value to the overall structure of the matrix, providing a foundation for understanding its low-rank properties.

First, the *effective rank* [30], defined as

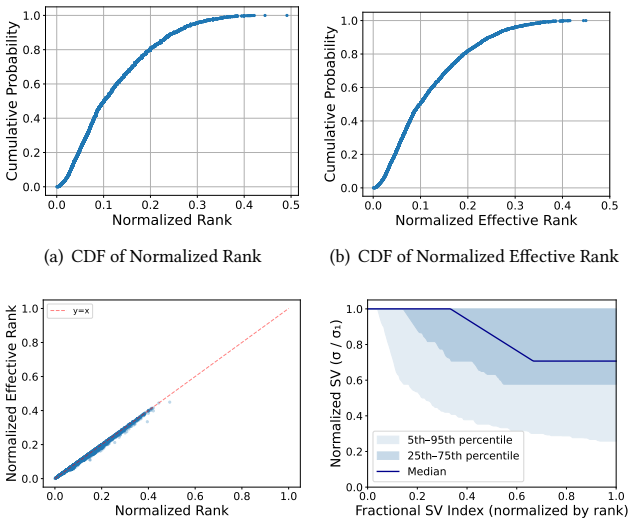
$$\text{erank}(\mathbf{A}) = \exp\Big(-\sum_i p_i \log p_i\Big), \quad \text{where } p_i = \frac{\sigma_i}{\sum_j \sigma_j}, \qquad (1)$$

provides a continuous scalar summary of how the singular values are distributed. It equals the matrix rank when all nonzero singular values are identical (maximally spread structure) and approaches 1 when a single singular value dominates (maximally concentrated structure). Second, the *singular value decay curve* visualizes the ordered singular values normalized by σ_1, illustrating how the spectral energy is distributed across the subspace.

Across the curated schema dataset in Table 4, the effective rank closely tracks the matrix rank. The median ratio erank/rank is 0.987, with 83.3% of schemas achieving an effective rank within 5% of their matrix rank. Pearson correlation between normalized rank and normalized effective rank is $r = 0.9986$ (We normalize the rank and the effective rank by the matrix dimension to enable comparisons across schemas of different sizes), as shown in Figure 1(c).

The cumulative distribution function (CDF) of normalized effective rank (Figure 1(b)) mirrors the normalized rank distribution, confirming that both measures yield consistent characterizations of the low-rank subspace. The SV decay percentile bands (Figure 1(d)) further demonstrate that 52.3% of schemas require all singular values to capture 90% of the total spectral energy.

These results provide two important confirmations for the matrix completion formulation. First, the effective rank corroborates the normalized rank analysis: join graph matrices are genuinely low-rank relative to their ambient dimension (median normalized rank ≈ 0.10), and this is not an artifact of a few dominant singular values inflating an otherwise noisy spectrum. Rather, every direction within the low-rank subspace carries meaningful relational structure. Second, the near-uniform distribution of singular values implies that the matrices are *well-conditioned* within their row/-column space (condition number $\sigma_1/\sigma_r \approx 1$). This is a favorable property for nuclear norm minimization: well-conditioned matrices satisfy standard incoherence assumptions more naturally [5], and the absence of extreme spectral gaps means that recovery is numerically stable: small perturbations in the observed entries do not disproportionately corrupt any singular component. In summary, the SVD analysis confirms that real-world join graphs occupy a low-dimensional, well-conditioned subspace, lending additional theoretical and empirical support to the low-rank matrix completion approach.



(a) CDF of Normalized Rank

(b) CDF of Normalized Effective Rank

(c) Correlation of Normalized Rank and Effective Rank

(d) SV Decay Percentile Bands

Figure 1: CDF of normalized rank, effective rank, and SV decay percential bands of the join graph matrices corresponding to the curated real-world database schemas.

Due to space constraints, Figures 1(b)-(d) are omitted from the main text; however, Table 4 reports the normalized effective rank statistics. We also summarize the SV decay curves using the *energy 90% index*, defined as the minimum number of singular values k required to capture 90% of the total spectral energy (i.e., $\sum_{i=1}^{k} \sigma_i^2 \geq 0.9 \sum_{j=1}^{r} \sigma_j^2$). To enable fair comparisons across diverse schema sizes, Table 4 presents this index normalized by the matrix rank.

The intuition behind the low-rank matrix completion formulation is that since real-world join graphs inherently exhibit high sparsity and a low-rank structure, using these properties as optimization objectives naturally guides the matrix completion process toward a matrix that accurately reflects real-world characteristics. We have also enhanced the motivation in the introduction.

Scalability and Various Statistics

To justify the scalability of our approach, we report the number of LLM calls made for different purposes and the total API costs (see Table 3), the join candidate pruning statistics (see Table 1) as well as the entity type caching statistics (see Table 2).

Our metadata-based pruning (e.g., data type compatibility and min-max value range checks) ensures a major percentage of invalid join candidates are removed (over 90% across four datasets and up to 99.79% on the REAL dataset as reported in Table 1). Using LLMs for predicting semantic column type matching further prunes a signiciant amount of true negatives as shown in Table 1.

Table 1: Pruning statistics of join candidates using metadata-based pruning and LLM-based pruning in the EM phase.

	Nexus Variant	TPC-H	TPC-DS	BIRD-SQL	REAL
# Join Candidates	Model Agnostic	1,830	90,100	324,415	667,590
# Pruned by Metadata		1647	86,749	308,789	666,175
(Pruning Rate)		(90.00%)	(96.28%)	(95.18%)	(99.79%)
# False Negatives		0	3	1	6
# Pruned in EM	XGBoost	174	3,084	15,484	1,275
(Pruning Rate)		(9.51%)	(3.42%)	(4.77%)	(0.19%)
	GPT-4o	181	3303	15,538	1,377
		(9.89%)	(3.67%)	(4.79%)	(0.20%)
# False Negatives	XGBoost	0	5	20	79
	GPT-4o	7	60	27	115

Precision-First Use Cases

The main paper reports the best F1 scores for each method while the associated precision values may not be the best observed. To address this concern, we add Figure 2 which shows how precision of Nexus-GPT-4o varies with respect to hyperparameters so users can configure a "precision-first" mode.

Overall, Nexus-GPT-4o maintains robust precision across four datasets, often favoring moderate to high regularization to minimize false positives in join graph inference. Specifically, $\lambda_1 \in \{1.0, 2.0\}$ yields the most stable precision on TPC-H (0.78) and BIRD-SQL (0.80–1.00). For TPC-DS, while precision reaches an absolute peak of 1.00 at the lowest $\lambda_1 = 0.1$, it remains consistently strong (up to 0.92) at $\lambda_1 = 2.0$. Similar to the quality trends observed in the F1 analysis (Section 4.5), the REAL dataset achieves its optimal precision (0.73) with a larger $\lambda_1 = 2.0$. This alignment suggests that for datasets with a lower normalized rank, stronger low-rank regularization is critical not just for overall quality, but specifically for ensuring that the predicted join relationships are accurate and well-grounded in the underlying data structure.

Cross-referencing heatmaps of precision and F1 scores reveals distinct sweet spots where Nexus achieves a high-quality balance. While a single "best of both worlds" setting is not always reachable, Nexus allows users to adjust the loss weights to prioritize specific goals. A configuration favoring precision (e.g., lower λ_1 in BIRD-SQL or TPC-DS) is suitable for high-stakes environments where false positives are costly, whereas a setting maximizing F1 (moderate λ_1) is preferable for comprehensive data exploration where a balance of accuracy and coverage is considered important.

LLM-in-the-Loop

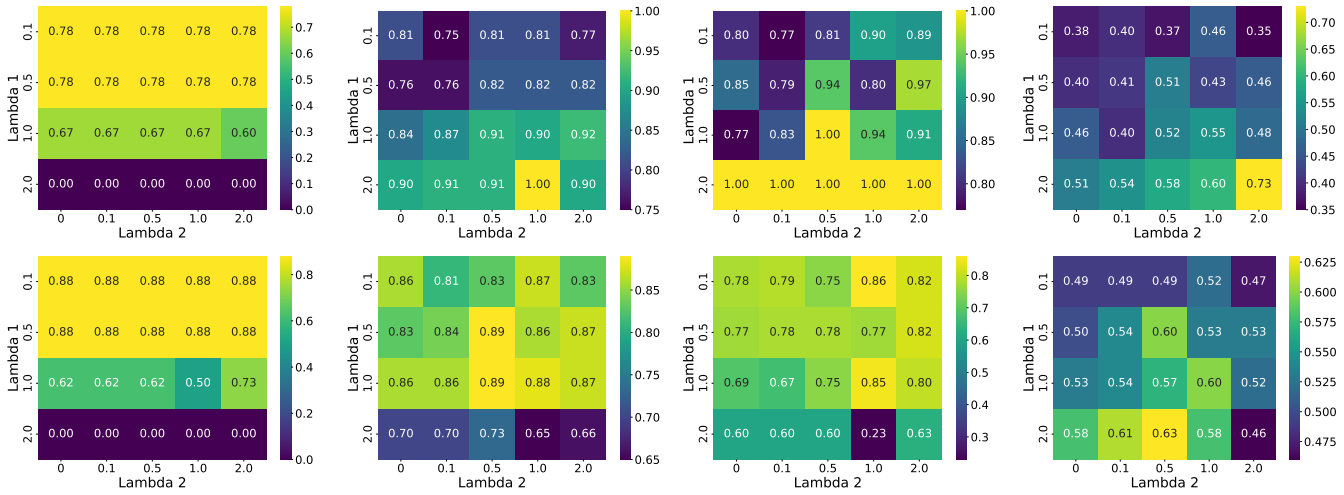
To address concerns regarding the LLM-in-the-loop, the supplementary material details the LLM prompts, hyperparameters and other settings.

To mitigate LLM hallucinations, we limit the response of each LLM call to $batch\_size * 30$ tokens and prompt the LLM to give the response in a specified JSON format. In case of API failures or LLMs giving "bad" response (e.g., returning only 9 entity types while batch size is 16), we make a retry up to three times, which is good enough for us to finish all experiments using a batch size of 16 without interruptions.

We report the number of LLM calls and total API costs in Table 3. When using XGBoost as the base model in Nexus, the LLM API costs are under \$0.5 on TPC-H, BIRD-SQL and REAL and under \$1.2 on TPC-DS. Even when using GPT-4o as the base model, the total API costs are under \$8 on each dataset, which is reasonable for the value provided by our approach in enterprise production environments. Moreover, the join candidate pruning strategy significantly reduces the number of LLM calls needed for join graph inference, which is key to keeping the monetary cost low.

Table 2: Caching statistics of column entity type annotation.

Dataset	Nexus Varient	Entity Type Caching		
		# Hits	# Misses	Hit Rate
TPC-H	XGBoost	83	15	84.69%
	GPT-4o	67	9	88.16%
TPC-DS	XGBoost	10,880	2774	79.68%
	GPT-4o	736	140	84.02%
BIRD-SQL	XGBoost	2,455	1,783	57.93%
	GPT-4o	374	224	62.54%
REAL	XGBoost	988	580	63.01%
	GPT-4o	480	220	68.57%



TPC-H **TPC-DS** **BIRD-SQL** **REAL**

Figure 2: Heat maps showing best-observed precision (first row) and F1 values (second row, copied from the original submission for easy cross-reference) of Nexus-GPT-4o w.r.t loss weights λ_1 and λ_2 in the optimization objective.

Table 3: LLM calls made in Nexus's join graph inference and their monetary costs across datasets.

Dataset	Nexus Variant	# LLM Calls for			Total # LLM Calls	Total $ Cost
		Join Prediction	Entity Type Annotation	Type Matching Prediction		
TPC-H	XGBoost	-	2	3	5	$0.01
	GPT-4o	12	2	1	15	$0.08
TPC-DS	XGBoost	-	21	424	445	$1.16
	GPT-4o	210	11	20	241	$1.64
BIRD-SQL	XGBoost	-	29	134	163	$0.44
	GPT-4o	977	18	18	1013	$7.54
REAL	XGBoost	-	20	46	66	$0.18
	GPT-4o	89	9	23	121	$0.80

NEXUS: Inferring Join Graphs from Metadata Alone via Iterative Low-Rank Matrix Completion

Tianji Cong
University of Michigan
Ann Arbor, Michigan, USA
congtj@umich.edu

Yuanyuan Tian
Microsoft Gray Systems Lab
Mountain View, California, USA
yuanyuantian@microsoft.com

Andreas Mueller
Microsoft Gray Systems Lab
Mountain View, California, USA
Andreas.mueller.ml@gmail.com

Rathijit Sen
Microsoft Gray Systems Lab
Redmond, Washington, USA
rathijit.sen@microsoft.com

Yeye He
Microsoft Research
Redmond, Washington, USA
yeyehe@microsoft.com

Fotis Psallidas
Microsoft Gray Systems Lab
New York, New York, USA
Fotis.Psallidas@microsoft.com

Shaleen Deep
Microsoft Gray Systems Lab
Madison, Wisconsin, USA
shaleen.deep@microsoft.com

H. V. Jagadish
University of Michigan
Ann Arbor, Michigan, USA
jag@umich.edu

Abstract

Automatically inferring join relationships is a critical task for effective data discovery, integration, querying and reuse. However, accurately and efficiently identifying these relationships in large and complex schemas can be challenging, especially in enterprise settings where access to data values is constrained. In this paper, we introduce the problem of join graph inference when only metadata is available. We conduct an empirical study on a large number of real-world schemas and observe that join graphs when represented as adjacency matrices exhibit two key properties: high sparsity and low-rank structure. Based on these novel observations, we formulate join graph inference as a low-rank matrix completion problem and propose NEXUS, an end-to-end solution using only metadata. To further enhance accuracy, we propose a novel Expectation-Maximization algorithm that alternates between low-rank matrix completion and refining join candidate probabilities by leveraging Large Language Models. Our extensive experiments demonstrate that NEXUS outperforms existing methods by a significant margin on four datasets including a real-world production dataset. Additionally, NEXUS can operate in a fast mode, providing comparable results with up to 6x speedup, offering a practical and efficient solution for real-world deployments.

1 INTRODUCTION

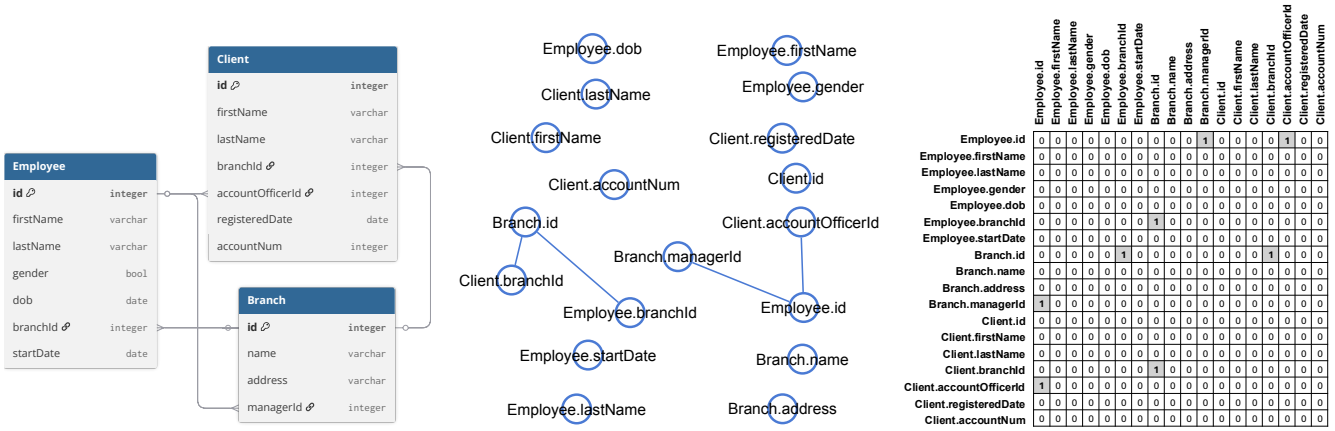
Detecting join relationships within a collection of tables is essential for data management tasks such as identifying primary and foreign keys (PK/FKs) in relational databases [6, 18, 29, 32], constructing Business Intelligence (BI) models for analytical workloads [1, 23], and finding related data in diverse sources like Web corpora [3, 4, 28, 31] and data lakes [2, 8, 11, 13–15, 33–35].

This capability is particularly important for modern lakehouse platforms. The very design principles that make the lakehouse architecture compelling (flexibility, scalability, and the unification of diverse data) create a challenging environment for join detection. Lakehouse operates on "loose schema", where referential constraints like PK/FKs are often unsupported or undeclared. Furthermore, the consolidation of disparate datasets into a single repository obscures the formal boundaries between databases or schemas. Consequently, inferring the complete set of join relationships, which we term the *join graph*, is a foundational capability in a lakehouse enabling automated data cataloging, enhanced data exploration, ML feature engineering, knowledge graph construction, and natural language querying. Figure 3(b) illustrates a join graph for a simple schema, where each node represents a table column and each edge indicates a join relationship between two columns.

Join detection is not a new problem, and it has been tackled by a variety of methods, ranging from traditional heuristics [6, 18, 26], embedding-based techniques [8, 14, 15], and ML models [1, 25, 29] to, most recently, large language models (LLMs) [12]. However, existing solutions predominantly rely on accessing data values. Utilizing data values—whether for direct value overlap calculation, embedding generation, or ML feature computation—is computationally expensive, especially for large datasets containing numerous and sizable tables. More importantly, the assumption that join detection algorithms can freely access data is often *unrealistic*. For data service providers operating under strict privacy and security constraints, accessing customer data is often infeasible. Beyond data dependence, existing solutions typically perform only "local" pairwise detection, ignoring the global join graph structure. While Auto-BI [23] addresses this limitation through global graph-based optimization, it is purpose-built for BI settings where schemas follow strict arborescent structures (e.g., snowflake schemas). This design assumption breaks down in lakehouse contexts, where schemas exhibit significantly looser, less constrained structures.

Unlike joinable table search [2, 8, 11, 13–16, 25, 34] which finds tables joinable with a given input table/column, we address a more challenging problem of *join graph inference* in this paper. Given a collection of tables (specifically, their schemas and metadata like column data types and min/max values), our objective is to infer

Conference'17, Washington, DC, USA
2026. ACM ISBN 978-x-xxxx-xxxx-x/YYYY/MM
https://doi.org/10.1145/nnnnnnn.nnnnnnn



(a) ER diagram (b) Join graph (c) Join graph matrix (adjacency matrix)

Figure 3: An example schema of a bank database with its corresponding join graph and join graph matrix.

the adjacency matrix of the corresponding join graph by uncovering equi-joins, particularly PK/FK relationships, among the tables. While these two problems are closely related, join graph inference is fundamentally harder: it requires discovering all pairwise join relationships across an entire table collection, resulting in quadratic worst-case computational complexity $O(n^2)$ with respect to the number of columns n, whereas joinable table search operates in linear worst-case time $O(n)$ by checking joinability against a specified input table/column. To this end, we develop Nexus, an *accurate* and *efficient* end-to-end join graph inference solution tailored for the enterprise setting. Nexus is designed to operate using *only metadata* (detailed in Section 2.1). Nevertheless, Nexus is flexible enough to incorporate *query logs* and data values to further improve accuracy when they are available (Section 4.6 and 4.7).

The design of Nexus is guided by the crucial insight that *real-world join graphs are highly structured*. Schemas are typically built around a small number of join keys, while most columns never participate in joins. This structure exhibits two key properties. First, the adjacency matrix of the join graph (or simply, the join graph matrix) is highly *sparse*, as illustrated in Figure 3(c). Second, besides those zero rows shown in Figure 3(c), the matrix exhibits significant redundancy; for example, columns `Employee.branchId` and `Client.branchId` share the same joinable column set, making their corresponding rows in the matrix identical. This redundancy implies that the join graph structure can be captured by a small number of latent factors. Mathematically, this suggests the join graph matrix has a *low rank* structure and can be approximated by a low-dimensional representation.

Our extensive analysis of 5,957 non-trivial real-world database schemas from the SchemaPile-Perm dataset [12] empirically confirms this insight: their join graphs are consistently sparse and low-rank. Around 98% of the databases have a density below 0.02, and over 95% have a normalized rank under 0.3. This study, detailed in Section 2.2, is to our knowledge the first and most extensive study of its kind and thus a significant contribution in itself. Establishing the high sparsity and low-rank nature of join graphs offers broader applicability than approaches restricted to strict arborescent structures common in BI settings [23]. In addition, our analysis informs our prioritization of detecting single-column joins. Among all databases containing join information in SchemaPile-Perm, only 7.5% contain multi-column joins, representing just 3.5% of all join cases. Consequently, while the current implementation of Nexus is optimized for this prevalent scenario, its design remains extensible to multi-column joins, as discussed in Section 3.7.

Based on our crucial insight, we formulate join graph inference as a *low-rank matrix completion* problem, where the goal is to recover unknown entries of the join graph matrix from a set of known ones under high sparsity and low-rank constraints. To achieve this, low-rank matrix completion acts as an intelligent inference engine. It predicts missing connections by leveraging the fact that real-world data connections are highly redundant rather than random, and are driven by just a few hidden, underlying patterns. By treating this inherent redundancy as an optimization objective, the algorithm bounds its search space, naturally guiding the recovery process toward a matrix that accurately reflects real-world schema characteristics. We initialize the matrix by assigning 0 to pruned column pairs and, where available, 1 to confirmed joins derived from query logs; while our method functions independently of query logs, they integrate naturally when present. For the remaining candidate entries that survive pruning, we utilize pretrained ML models or LLMs to score their likelihoods of being true joins. We then introduce a novel objective function that augments the standard low-rank completion goal with two additional terms: (1) a loss term that encourages the recovered entries to align with the probability scores given by ML models/LLMs, and (2) a regularization loss term that promotes the sparsity observed in real-world join graphs. This formulation allows us to effectively leverage predictions from pretrained models while adhering to the inherent structural properties of the join graph.

In practice, pretrained models often struggle to generalize to unseen datasets, and forcing latent entries to conform to inaccurate predictions from these models can significantly degrade the quality of the inferred join graph. To address this challenge, we propose an

Expectation-Maximization (EM) algorithm that iteratively refines the low-rank matrix completion by incorporating semantic guidance from LLMs. The EM algorithm alternates between two steps: (1) performing low-rank matrix completion, and (2) updating the prior probability estimates of join candidates using an LLM. A key component of this approach is leveraging an LLM to assess the semantic compatibility of *column entity types*, which is a prerequisite for valid joins. We define a column entity type as a free-text annotation that encapsulates the semantic meaning of a column's values within the context of its table (e.g., annotating an `ID` column in an `Employee` table as "employee ID"). Unlike existing annotation methods that depend on fixed sets of entity types or taxonomies, Nexus utilizes LLMs to handle diverse column semantics. We leverage LLMs to generate dynamic entity annotations and determine their semantic compatibility. Based on this assessment, we adjust the join probability scores—upweighting strong semantic matches and penalizing mismatches. This iterative refinement enables Nexus to correct inaccurate initial predictions and adapt to the unique semantics of unseen datasets.

To summarize, this paper makes the following contributions:

- We propose Nexus, an accurate and efficient end-to-end solution for join graph inference, tailored for privacy-conscious enterprise settings. By relying on metadata only, and optionally leveraging query logs when available, Nexus operates effectively without direct access to data values.
- Our extensive empirical analysis of nearly 6K real-world database schemas confirms our crucial and novel insight: join graphs are both sparse and consistently low-rank. This new discovery forms the foundation for our novel join graph inference algorithm (Section 2).
- We formulate join graph inference as a low-rank matrix completion problem, with an optimization objective that explicitly exploits observed sparsity and low-rank structure while using pretrained models for initial join likelihoods (Section 3).
- To overcome the challenge of poor prior predictions from pretrained models, we propose a novel EM algorithm that alternates between performing low-rank matrix completion and leveraging an LLM to iteratively refine the probability scores of join candidates (Section 3.5).
- We demonstrate the effectiveness and efficiency of Nexus on four datasets including a real production dataset. Our metadata-only approach significantly outperforms baselines including those using data values (Section 4).

2 PRELIMINARIES AND OBSERVATIONS

This section establishes the foundation for our join graph inference solution. We will start by introducing key concepts and formally defining the problem. Following that, we will present findings from an empirical study on a large number of real-world database schemas, which directly motivate the design of our algorithm.

2.1 Problem Statement

We first give the definitions of join graphs, join graph matrices, and join graph probability matrices. Figure 3 shows the join graph and join graph matrix for a simple example database schema.

Definition 1 (Join Graph)**.** Given a set of tables T, a join graph is an undirected graph $G(T) = (V, E)$ where:

- $V = \{c_1, c_2, ..., c_n\}$ represents the set of all columns from the tables in T.
- E is the set of join relationships. An edge (c_i, c_j) exists between two columns c_i and c_j, if and only if a join relationship exists between them.

When the context is clear, we will refer to the join graph $G(T)$ simply as G.

Definition 2 (Join Graph Matrix)**.** Let $G = (V, E)$ be the join graph for a set of tables T. The join graph matrix $A(T) \in \{0, 1\}^{n \times n}$, or simply denoted as A when there is no ambiguity, is the adjacency matrix of the join graph G:

$$A_{i,j} = \begin{cases} 1 & \text{if } (c_i, c_j) \in E \\ 0 & \text{otherwise} \end{cases} \tag{2}$$

Since we consider the join graph undirected, the resulting matrix A is symmetric ($A_{i,j} = A_{j,i}$) with a zero diagonal ($A_{i,i} = 0$).

Definition 3 (Join Graph Probability Matrix)**.** Let $G = (V, E)$ be the join graph for a set of tables T. The join graph probability matrix $S(T) \in [0, 1]^{n \times n}$, or simply denoted as S in case of no ambiguity, is an $n \times n$ matrix, where each entry $S_{i,j}$ denotes the probability that column c_i and column c_j are joinable.

S is also a symmetric matrix ($S_{i,j} = S_{j,i}$) with a zero diagonal ($S_{i,i} = 0$) due to the undirected nature of join graphs.

Definition 4 (Metadata-Only Join Graph Inference)**.** The problem of *metadata-only join graph inference* is to reconstruct the join graph matrix A for a given set of tables T, using *only metadata* of T.

The metadata we consider in this work includes table names, column names, column data types, and simple column statistics (total number of values, number of distinct values, number of nulls, and minimum and maximum values). These basic statistics are commonly available in the catalog service of a lakehouse platform for the purpose of query optimization. We simply reuse them for join graph inference. Critically, and in line with enterprise privacy and security requirements, our approach assumes no access to the actual data values within the columns. While we target the metadata-only setting, handling obfuscated schemas is out of the scope of this work.

Our primary goal is to detect 1:1 and N:1 relationships, which correspond to PK/FK joins. Many-to-many (N:N) relationships can then be inferred via transitive closure. While we focus on single-column joins, the method is extensible to multi-column joins, as discussed in Section 3.7.

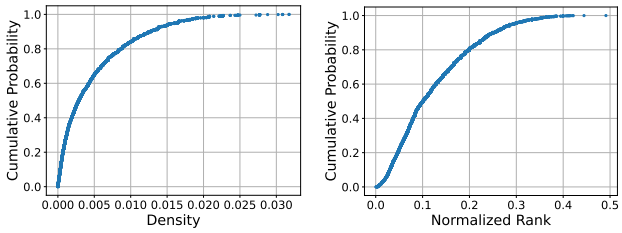
2.2 Empirical Analysis

To validate our hypothesis of join graph structure, we conducted an extensive empirical study to understand the characteristics of real-world join graphs.

For our empirical study, we curated a dataset of realistic, non-trivial database schemas from the SchemaPile-Perm collection [12], a GitHub-crawled corpus that reflects real-world schema diversity across a wide range of domains, SQL dialects, and structural properties. Starting with 22,989 databases, we first filtered this dataset

Table 4: Statistics of the curated database schemas.

Statistics per database	Min	Max	Mean	Median
# tables	4	977	15	9
# columns	17	4572	108	56
Avg. # columns / table	4.04	37.46	6.73	6.00
Density	1.56×10^{-7}	0.032	0.005	0.003
Norm. rank	5.58×10^{-4}	0.491	0.124	0.1
Norm. effective rank	5.58×10^{-4}	0.448	0.121	0.098
Norm. energy 90% index	0.333	1.0	0.909	1.0



(a) CDF of Density (b) CDF of Normalized Rank

Figure 4: CDF of density and normalized rank of the join graph matrices for the curated real-world database schemas.

to the 11,809 databases (around 51%) that contain join relationships. We then focused on single-column joins by excluding the 7.5% of schemas with multi-column relationships. Finally, we filtered out trivial databases, removing those with three or fewer tables or an average of four or fewer columns per table. This process resulted in a focused dataset of 5,957 databases for our analysis.

As detailed in Table 4, this curated dataset represents a large and diverse collection of real-world database schemas (from small transactional to large enterprise-scale databases): the number of tables per database ranges from 4 to 977, and the number of columns per database ranges from 17 to 4572. Besides basic statistics, our analysis focused on two key properties of join graph matrices: *sparsity* and *rank* (see Figure 4).

- *High Sparsity:* We measured sparsity using matrix density, defined as $\frac{\#\text{ nonzeros}}{n^2}$ for an $n \times n$ matrix. The results confirm that real-world join graphs are extremely sparse, with an average density below 0.005 and a median density under 0.003. As shown in Figure 4(a), about 98% of databases having a density less than 0.02.
- *Low-Rank Structure:* The *rank* of an $n \times n$ matrix A, denoted as $\text{rank}(A)$, measures the number of linearly independent rows (or columns) in A, intuitively capturing its structural complexity. A high rank (close to n) indicates a complex matrix with completely linearly independent rows/columns, while a low rank denotes structural redundancy where a few basic patterns can represent the entire matrix. Computationally, rank is determined by counting the non-zero singular values via Singular Value Decomposition (SVD). We quantify this property of join graphs by using the *normalized rank*, which is defined as $\frac{\text{rank}(A)}{n}$. This metric indicates how close a matrix is to being full-rank. We found that the average and median normalized ranks were only 0.12 and 0.10, respectively. As Figure 4(b) illustrates, over 95% of the databases have a normalized rank below 0.3. To further characterize this low-rank structure, we analyze the singular value (SV) distribution using *normalized effective rank* [30] and *SV decay curves*, summarized by the *normalized energy 90% index* (the minimum singular values k needed to capture 90% of the spectral energy, $\sum_{i=1}^{k} \sigma_i^2 \geq 0.9 \sum_{j=1}^{r} \sigma_j^2$, normalized by the matrix rank). As Table 4 shows, these metrics confirm that real-world join graphs are genuinely low-rank relative to their ambient dimensions. This is not merely an artifact of a few dominant singular values; rather, every direction within the low-rank subspace captures meaningful relational structure.

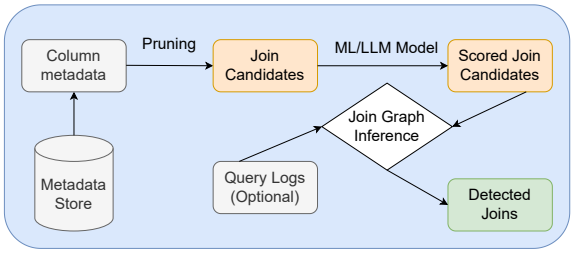


Figure 5: Overview of Nexus pipeline that infers join graphs from only metadata.

These two key observations, high sparsity and low-rankness of join graphs, confirms our intuition that most columns in a schema are not joinable and that the overall join structure contains significant redundancy, allowing it to be represented in a low-dimensional space. In the following section, we describe our novel join graph inference algorithm based on these two key observations.

3 Nexus Design

In this section, we provide an overview of our join graph inference solution, Nexus, and detail each core system component.

3.1 Overview

Figure 5 illustrates the overall workflow of Nexus. The process begins by retrieving schema information and column-level metadata for a given collection of tables from the metadata store. These metadata are then used to prune unlikely join column pairs, significantly reducing the number of potential join candidates. Next, each candidate is scored using a pre-trained model, which can be either an ML model or an LLM. These scored candidates, optionally supplemented with known joins extracted from query logs, form a join graph probability matrix. In this matrix, non-candidate pairs are assigned a probability of 0, known joins from query logs are given a probability of 1.0, and the remaining entries are populated with scores from the pre-trained model. The probability matrix is subsequently fed into a matrix completion algorithm, which is based on our observations of high sparsity and low-rank structure within the join graph matrix. This algorithm is further enhanced by an iterative Expectation-Maximization (EM) procedure designed to mitigate any inaccuracies from the pre-trained model. The output is the final inferred join graph matrix, which identifies the detected join relationships.

3.2 Metadata and Join Candidates Pruning

Exhaustively evaluating every pair of columns as a potential join candidate incurs a quadratic cost relative to the number of columns, making it impractical for large datasets with thousands of columns. To address this, we collect column-level metadata from the metadata

stores and apply cheap filters to eliminate a large number of unlikely join candidates early on.

We begin by discarding columns that are unlikely to serve as join keys, such as nested types (e.g., `JSON`, `ARRAY`, `MAP`), measure types (e.g., `FLOAT`, `DOUBLE`, `DECIMAL`), as well as `BLOB` and `BOOLEAN` types. For the remaining columns, we leverage the data type and the basic statistics from the metadata store, including cardinality (number of unique values), null value ratio, and min/max values. These statistics are typically maintained in metadata stores for query optimization purposes. Column pairs with incompatible data types (e.g., INTEGER vs. VARCHAR) are pruned immediately. Additionally, when inferring N:1 or 1:1 joins, we apply further pruning criteria, including ensuring uniqueness on the "1" side, verifying that the cardinality on the "N" side does not exceed that of the "1" side, and confirming that the value domain of the "N" side is a subset of the "1" side. These pruning criteria can be implemented in SQL syntax and executed efficiently in database systems.

While more sophisticated profiling and pruning techniques exist [18, 19, 21], they typically incur higher computational costs. We opt for these simpler, more efficient metadata-only filters because they effectively eliminate a large proportion of true negatives with minimal overhead.

3.3 Scoring Join Candidates

The join candidates produced by the initial pruning step may still contain many false positives. For instance, a user ID column and a product ID column may share compatible data types and value ranges, yet they are not semantically joinable. Prior works [1, 23, 29] have typically employed ML models that combine multiple signals, such as embedding similarities of column names and Jaccard similarities of column values, to predict join relationships and estimate the associated probability scores. However, prior works rely on computing ML features that require access to the actual data values. Such computations are expensive and become prohibitive for large tables, often containing millions or billions of rows in enterprise environments. In contrast to most prior work, we assume no access to data values. We employ two metadata-based strategies to estimate the probability scores of join candidates:

- *Pre-trained ML Model:* We pre-train an ML model exclusively on metadata-derived features. These include pairwise features derived from column names (max containment, Jaccard similarity, the ratio of edit distance to max length, Jaro-Winkler similarity, and the cosine similarity of weighted embeddings), and column-wise features (data type encodings, cardinality ratios, and null-value number ratios).
- *LLM:* We can leverage an LLM as a pre-trained model, prompting it directly for both predictions and associated confidence levels, categorized as low, medium, and high. These qualitative confidence levels are then mapped to predefined probability scores (e.g., low $\rightarrow$ 0.1, medium $\rightarrow$ 0.6, and high $\rightarrow$ 0.9). This indirect elicitation of confidence is motivated by the known limitations of LLMs in generating reliable, well-calibrated confidence scores [9, 24, 27].

After this step, we construct a join graph probability matrix using the predicted scores from the pre-trained ML or LLM model, optionally augmented with confirmed join column pairs extracted from query logs, if available.

Despite the usefulness of pretrained models, they face two significant limitations: (1) their predictions are inherently local, considering only the features of individual column pairs and ignoring the global structure of the join graph, and (2) models trained on one dataset may not generalize effectively to others, and their confidence scores can be misleading. We address these critical challenges in Sections 3.4 and 3.5, respectively.

3.4 Join Graph Inference through Low-Rank Matrix Completion

After initial pruning and scoring, we obtain a join graph probability matrix, S. This matrix is partially observed: some entries are considered certain, with pruned candidates having value 0 and known joins from query logs having value 1, while the rest hold probability scores from a pre-trained model. We define Ω as the set of indices for the certain entries in S: $\Omega = \{(i, j) \mid (c_i, c_j)$ are either pruned out or from query logs$\}$.

The objective of join graph inference is to recover a matrix $M \in [0, 1]^{n \times n}$ that is a high-quality approximation of the unknown ground truth join graph matrix A. This recovery process only affects the uncertain entries of S. Drawing from our key observations that real-world join graphs are highly sparse and low-rank, we formulate this inference task as a low-rank matrix completion problem.

The standard formulation of low-rank matrix completion seeks to minimize the rank of M while preserving the observed values. Formally, the optimization objective can be written as:

$$\begin{aligned} &\text{minimize} \quad \text{rank}(M) \\ &\text{subject to} \quad M_{i,j} = A_{i,j} \text{ for } (i, j) \in \Omega \end{aligned} \tag{3}$$

where $\text{rank}(\cdot)$ gives the rank of a matrix.

This is a non-convex optimization problem due to the rank function and is known to be NP-hard [7]. A well-established convex relaxation approximates the rank function with the nuclear norm of the matrix [5], resulting in the following optimization objective:

$$\begin{aligned} &\text{minimize} \quad ||M||_* = \sum_{k=1}^{n} \sigma_k(M) \\ &\text{subject to} \quad M_{i,j} = A_{i,j} \text{ for } (i, j) \in \Omega \end{aligned} \tag{4}$$

where $\sigma_k(\cdot)$ denotes the k-th largest singular value of a matrix and the nuclear norm $||\cdot||_*$ is the sum of all singular values.

While alternative approaches to low-rank matrix completion exist, such as matrix factorization [17, 20], they require pre-specifying the rank of the matrix to be recovered. Such prior knowledge of the matrix rank is not readily available in our setting. On the other hand, nuclear norm relaxation is generally considered less scalable than matrix factorization, as it needs to estimate all the parameters corresponding to the latent entries, which increase quadratically with respect to the matrix dimension. However, this concern is largely mitigated in our context due to a crucial observation: real-world join graphs are extremely sparse. Specifically, we observed that the detected join candidates (after pruning) constitute less than 5% of the entries in the matrix we need to recover. Consequently, the number of parameters in our optimization is dramatically reduced, making the nuclear norm approach both feasible and efficient. This key insight not only addresses the scalability issue but also motivates us to explicitly incorporate a sparsity-inducing constraint

into our recovery objective, ensuring the final solution reflects this fundamental property of join graphs.

Additionally, we have access to the confidence scores for join candidates in the join graph probability matrix S, provided by the pre-trained model. These scores serve as prior information to guide the recovery of the latent entries in M.

Let $\bar{\Omega}$ denote the complement of Ω, which contains the set of positions for the uncertain entries in our probability matrix S. To integrate the observed sparsity and the prior beliefs, we propose the following optimization objective:

$$\begin{aligned} &\text{minimize} \quad ||\mathrm{P}_{\bar{\Omega}}(S-M)||_F^2 \;+\; \lambda_1\,||M||_* \;+\; \lambda_2\,||M||_1 \\ &\text{subject to} \quad M_{i,j} = A_{i,j} \text{ for } (i,j) \in \Omega \end{aligned} \tag{5}$$

Here, $\mathrm{P}_{\bar{\Omega}}$ is a projection operator that preserves the entries with positions in $\bar{\Omega}$ while zeroing out all others. $||\cdot||_F$ and $||\cdot||_1$ denote the Frobenius norm and the 1-norm of a matrix, respectively.

The above object function can be broken down into the following three components:

- *Confidence Score Regularization*: The term $||\mathrm{P}_{\bar{\Omega}}(S-M)||_F^2$ encourages the final matrix M to align with the confidence scores from the pre-trained model, but only for the uncertain entries in $\bar{\Omega}$.
- *Low-Rank Regularization*: The term $\lambda_1\ ||M||_*$ uses the nuclear norm to encourages the solution M to be low-rank, aligning with our key observation.
- *Sparsity Regularization*: The term $\lambda_2\ ||M||_1$ uses the L1 norm to promote sparsity in M, effectively pushing the probabilities of unlikely joins towards zero.

λ_1 and λ_2 are hyperparameters weighting the regularization power of the low-rank term and the sparsity term, respectively.

Note that the recovered matrix M contains real values in $[0,1]$, rather than binary indicators. To produce the final binary decisions, we apply a threshold θ to the latent entries of M. The resulting decision matrix $\hat{M} \in \{0,1\}^{n\times n}$ is defined as:

$$\hat{M}_{i,j} = \begin{cases} M_{i,j} & \text{if } (i,j) \in \Omega \\ \mathbf{1}(M_{i,j} \geq \theta) & \text{otherwise} \end{cases} \tag{6}$$

where $\mathbf{1}(\cdot)$ is the indicator function.

3.5 Iterative Join Graph Inference via Expectation-Maximization Algorithm

As shown in Equation 5, we (partially) align the recovered matrix with the probability matrix determined by a pre-trained ML/LLM model. This approach assumes that the model accurately estimates the probability scores of join candidates. However, this assumption may not hold when applying a pretrained model to unseen datasets with very different distributions from the training datasets. Consequently, forcing latent entries in the recovered matrix to match potentially inaccurate estimates can degrade result quality. To address this issue, we propose an Expectation-Maximization (EM) algorithm (Algorithm 1), which iteratively improves the quality of low-rank matrix completion using guidance from an LLM.

EM is a widely adopted approach in ML for computing maximum likelihood estimates in models with incomplete or latent variables [10]. The algorithm, outlined in Algorithm 1, operates iteratively in two main steps. The *Expectation step* (E-step) involves low-rank matrix completion to estimate the values of latent entries.

Algorithm 1: Expectation-Maximization Algorithm for Iterative Join Graph Inference

Input : $S^{(0)}$, initial probability matrix;
$\Omega^{(0)}$, initial set of known certain entries in $S^{(0)}$;
Γ, max number of iterations;
ϵ, tolerance for early stopping
Output : M, recovered matrix

```
/* Map from column to its entity type (cache)        */
col_entity_type = {};
for t = 0, 1, ..., Γ − 1 do
    /* E-step: Recover the latent matrix M^(t+1)     */
    M^(t+1) = low_rank_matrix_completion(S^(t), Ω^(t));
    /* Check for convergence                         */
    if t > 0 then
        /* No candidates left or last iteration      */
        if Ω̄^(t) == ∅ or t == Γ − 1 then
            break;
        end
        if ||M^(t+1) − M^(t)||_F ≤ ε then
            break;
        end
    end
    /* M-step: Update the probability matrix         */
    S^(t+1), Ω^(t+1), col_entity_type =
      update_prob_matrix(M^(t+1), Ω^(t), col_entity_type)
end
return M^(t+1)
```

This estimation is based on the observed data Ω and the probability scores $S^{(t)}$ given by an ML/LLM model. This E-step can be interpreted as determining the posterior probabilities of the latent entries given the observed data and the prior knowledge from the ML/LLM model. Subsequently, in the *Maximization step* (M-step), we refine the probability matrix by incorporating a semantic compatibility assessment of column entity types using an LLM. Specifically, we harness the semantic power of LLMs to enforce a key condition for valid joins: *two columns that can be joined must have semantically equivalent entity types.*

We define a *column entity type* as a free-text annotation that captures the specific semantics which a column represents within its relational table. For instance, an ID column in a table of food orders should ideally be annotated as "food order ID", whereas an ID column in a table of users should be annotated as "user ID". Leveraging the advanced semantic understanding capabilities of LLMs, we employ an LLM to annotate column entity types and to determine the semantic equivalence between two entity types. The match of entity types constitutes a necessary condition for valid joins and substantially reduces the number of spurious join candidates (e.g., two ID columns have coincidental value overlap but refer to distinct types of entities). Critically, our approach does not rely on a fixed vocabulary of entity types, as column semantics can vary across different tables and databases, and no existing entity type vocabularies have a comprehensive coverage of the nuanced spectrum of column semantics.

Algorithm 2: Update the Probability Matrix (**update_prob_matrix**)

Input : S, the probability matrix;
Ω, the set of known certain entries in S;
col_entity_type, column to entity type map;
$low\_threshold$, the threshold to prompt LLM for entity types and type match check;
$high\_threshold$, the threshold to promote a latent entry to be a hard positive;
δ, the penalty factor for entity type mismatch

Output : updated S, Ω, and col_entity_type

$join\_candidates = \{(i,j) \mid (i,j) \in \bar{\Omega} \text{ and } i < j\}$;

/* Loop through join candidates */

for $(i,j) \in join\_candidates$ **do**
  if $S[i][j] < low\_threshold$ **then**
    continue;
  end
  if $i \notin col\_entity\_type$ **then**
    $col\_entity\_type[i] =$ **prompt_llm_for_entity_type**(col_i);
  end
  if $j \notin col\_entity\_type$ **then**
    $col\_entity\_type[j] =$ **prompt_llm_for_entity_type**(col_j);
  end
  /* Boost probability of type-matched entry */
  if $col\_entity\_type[i] == col\_entity\_type[j]$ **or** **prompt_llm_for_soft_type_match**(col_i, col_j) **then**
    if $S[i][j] \geq high\_threshold$ **then**
      $S[i][j] = S[j][i] = 1.0$;
      /* Add new known entries */
      $\Omega.add(\{(i,j),\ (j,i)\})$;
    else
      $S[i][j] = S[j][i] = high\_threshold$;
    end
  else
    /* Decay probability of unmatched entry */
    $S[i][j] *= \delta; S[j][i] *= \delta$;
  end
end
return $S, \Omega, col\_entity\_type$

As detailed in Algorithm 2, the M-step iterates through latent entries (join candidates) and prompts an LLM to identify the entity types of columns based on column and corresponding table names. For each candidate pair with a score exceeding a predefined *low threshold*, we prompt an LLM to infer their entity types if they do not have one. To reduce the computational and monetary cost of LLM calls, we cache the identified entity types. Subsequently, we conduct a character-wise comparison of their entity types. If this comparison fails, we further prompt the LLM for a soft check of whether the entity types match semantically. The alignment of entity types is a prerequisite for a valid join. We have observed LLMs to be particularly effective in performing such nuanced semantic comparisons. If the entity types align and the associated probability score exceeds a predefined *high threshold*, we promote the pair to be a hard positive, setting its probability score to 1 and removing it from the set of latent entries. If the types match but the probability score falls below the high threshold, we boost the probability score to this threshold value, ensuring the pair undergoes another LLM check in the next EM iteration. This design accounts for the low initial probability estimate of the pair from the pre-trained model and the inherent stochasticity of LLM outputs. If the entity type check fails, we penalize the pair by reducing its probability score by a specific factor δ, which we set to 0.5 for all experiments. In the implementation, to further reduce the latency of LLM calls, we batch multiple requests in one prompt with a batch size of 24.

The iterative process of the EM algorithm continues until either a predefined maximum number of iterations, Γ, is reached or convergence is achieved. The convergence is determined by the change in the recovered matrix (quantified by the Frobenius norm) falling below a specified tolerance, ϵ. We set ϵ to $1e^{-5}$ in all experiments.

For latency-sensitive scenarios, our system can be configured to perform a single iteration of the EM algorithm (only low-rank matrix completion). This fast mode, which we call Nexus-Fast, is particularly useful when pre-trained models provide strong priors. It bypasses all LLM calls in the EM algorithm, ensuring maximum inference speed while still improving over the pretrained models and achieving competitive results compared to our full EM algorithm. In the following text, we refer to our full EM algorithm as Nexus-EM to distinguish from Nexus-Fast.

3.6 Optimization via Core Submatrix Reduction

The procedure described above recovers a matrix M of size $n \times n$, where n is the total number of columns in the table collection. For a large table collection, n can be very big. However, since our primary goal is to infer latent entries determined by join candidates that typically involve only a small subset of all columns, we can instead recover a smaller matrix, which we call the *core submatrix*:

Definition 5 (Core Submatrix)**.** Given a symmetric $n \times n$ matrix M, the core submatrix M' is defined as the submatrix obtained by removing all rows and columns from M that contain only zero entries. Formally, let $I = \{i \mid \exists j, M_{i,j} \neq 0\}$ be the set of indices corresponding to non-zero rows (and equivalently columns, since M is symmetric). Then, the core submatrix can be denoted as $M' = M[I, I]$. The dimension of M' is $n' \times n'$, where $n' = |I|$.

Since most columns are not join keys, especially those measure-oriented columns ubiquitous in analytical workloads, typically the core submatrix M' is much smaller than M (i.e. $n' << n$). Importantly, our approach based on low-rank matrix completion remains applicable, as we found that M' also exhibits high sparsity and a low-rank structure akin to the full adjacency matrix.

Recovering a smaller core submatrix M' offers significant practical benefits. Empirically, its dimension is often less than half the size of M, resulting in faster runtime and reduced memory usage. The main caveat is that the component of pruning join candidates is not perfect, and some relevant join keys may be excluded from M' and not recovered. Through experiments, we observe that this optimization gives almost 25% speedup when only running low-rank matrix completion (i.e., Nexus-Fast), so we enable this optimization by default for Nexus-Fast. For Nexus-EM, this optimization is less significant as the main performance bottleneck lies in LLM calls

Table 5: Statistics of experimental datasets.

Datasets	# Schema	# Tables	# Columns	Density	Normalized Rank
TPC-H	1	8	61	0.004	0.20
TPC-DS	1	24	425	0.001	0.08
BIRD-SQL	11	75	806	0.003	0.10
REAL	3	30	1156	0.001	0.02

(from a remote service). Hence, we choose to recover the full matrix without losing any relevant join keys.

3.7 Extension to Multi-Column Joins

While we have discussed our approach in the context of single-column joins, it's easily adaptable for multi-column join scenarios. The core idea remains the same: we treat every set of k columns within a table as a single conceptual unit. These units are then subject to the same pruning rules outlined in Section 4.5.1. This extension leads to a larger probability matrix for the low-rank matrix completion task. Specifically, for a set of tables T, the matrix dimension becomes $m \times m$, where $m = \sum_{\tau \in T} \binom{n_\tau}{k}$ and n_τ is the number of columns in table τ. Each entry in this matrix represents the join probability of a pair of k-column units.

Crucially, this expanded matrix also exhibits high sparsity and a low-rank structure. This allows us to directly apply the same matrix completion and EM algorithms (Section 3.4 and Section 3.5) to detect multi-column joins. In fact, the presence of multiple columns in a unit increases the matrix size while the overall number of multi-column joins is typically smaller. This often leads to an even higher sparsity and lower rank in the join graph matrix, which can benefit our algorithms. Additionally, the core submatrix reduction optimization from Section 3.6 is also applicable and can be employed to further improve the efficiency.

Despite this capability, our analysis of the SchemaPile-Perm dataset shows that multi-column joins constitute only a small fraction (7.5%) of real-world schemas. Consequently, our experimental evaluation focuses on the more prevalent single-column joins.

4 EXPERIMENTS

This section presents our experimental results, covering quality and efficiency comparisons as well as a hyperparameter ablation study. Unless otherwise specified (as in Section 4.7), all experiments use the metadata-only setup.

4.1 Experiment Setup

Datasets. Table 5 lists the datasets used to evaluate Nexus, covering both standard benchmarks and complex multi-source scenarios. We include TPC-H and TPC-DS, following the state-of-the-art (SOTA) work [23]. Since these benchmarks represent single-schema environments, we also created a multi-source scenario by combining 11 databases from the development set of BIRD-SQL [22]. Additionally, we introduce REAL, a production dataset spanning three distinct sources where only metadata and query logs are available. For ground truth, we rely on defined PK/FK constraints for the synthetic datasets and derive relationships from query logs for REAL, as it lacks explicit constraints. REAL reflects the "messiness" of production settings, providing a rigorous test for join detection. Although we use the Schemapile-Perm dataset to analyze join graph structure, we cannot use it for evaluation because it lacks the actual data values and column statistics. Instead, we utilize this dataset for ML model training (see below). Finally, column 5 of Table 5 reports the worst-case number of candidate pairs, reflecting the growth in computational complexity as the dataset size increases.

Baselines. First, we evaluate standalone ML models: XGBoost and GPT-4o. For XGBoost, we trained on a large corpus of schemas from SchemaPile-Perm (Section 2.2), utilizing a superset of metadata features from [1]. For GPT-4o, we used zero-shot prompts with metadata. These two models also serve as priors for our system (denoted Nexus-XGBoost and Nexus-GPT-4o).

Second, we compare against SemPy [26] (from Microsoft Fabric) and Auto-BI (the current SOTA). In addition to their original versions that utilize data values, we adapted them for our primary metadata-only evaluation. For SemPy, we disabled value-based overlap checks, relying on column name similarity (Ratcliff/Obershelp) and other heuristics. For Auto-BI, which frames join detection as a graph optimization problem, we replaced its original XGBoost model with our XGBoost model trained on metadata features.

Finally, we include Aurum [15] and DeepJoin [14]. These methods focus on the related "joinable table search" problem and primarily rely on data values; therefore, we restrict their evaluation to the experiments where data access is permitted (Section 4.7). We also attempted to evaluate JOSIE [34] but excluded it as it consistently timed out (>1 hour for multi-source datasets), a limitation also noted in prior work [11].

Metrics. Due to the class imbalance in detected join candidates, we evaluate the quality of all approaches using F1 score, precision, and recall. The reported numbers represent the best results for each method after we performed best-effort tuning on each dataset. For efficiency comparison, we report runtime in seconds.

Hardware. All experiments were conducted on an Azure virtual machine (standard NC12s v3) with 12 vCPUs and 224 GiB of RAM.

4.2 Quality Comparison

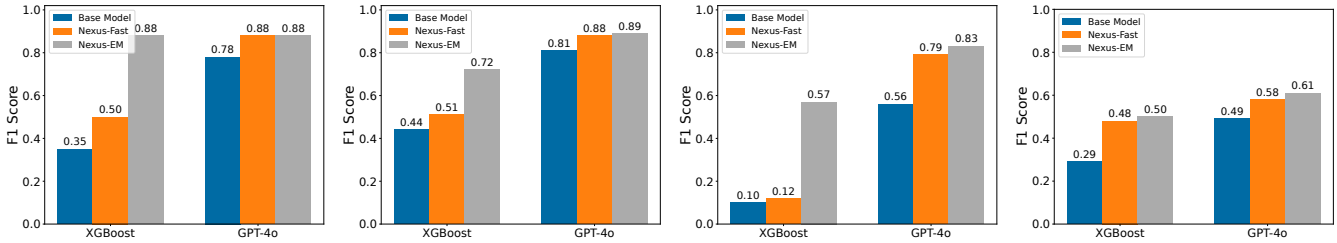
Table 6 presents the F1 score, precision and recall for all metadata-only approaches across the four datasets. Nexus consistently outperforms all baselines by a significant margin. For instance, Nexus-GPT-4o (Nexus with GPT-4o priors) surpasses Auto-BI, the previous SOTA, by 32, 37, 51, and 53 F1 points on the TPC-H, TPC-DS, BIRD-SQL and REAL datasets, respectively. Similarly, SemPy lags behind Nexus-GPT-4o by 47, 46, 54, and 35 F1 points on these datasets.

XGBoost, pretrained on the SchemaPile-Perm dataset with only metadata features, struggles to generalize to the test datasets, yielding the lowest F1 scores on TPC-H and BIRD-SQL. In contrast, even with these weak priors, Nexus-XGBoost significantly improves the F1 score by 53 points on TPC-H, 28 on TPC-DS, 47 on BIRD-SQL, and 21 on REAL. This substantial improvement is attributed to Nexus's iterative optimization (the EM algorithm in Algorithm 1), which leverages the low-rank structure of join graphs and LLM-based column entity type checks.

Nexus also improves upon GPT-4o priors, even when the model shows strong out-of-the-box results. GPT-4o's strong result quality on the public TPC-H and TPC-DS benchmarks is expected, as it likely memorized their join ground truth during training. While BIRD-SQL is also public, combining all 11 databases into a single dataset causes GPT-4o to generate a non-trivial number of false

Table 6: Quality comparison of metadata-only approaches on four datasets. Sempy and Auto-BI are adapted to use only metadata for comparison. Section 4.7 gives the comparison where data values are available and baselines operate in their full shape.

	TPC-H			TPC-DS			BIRD-SQL			REAL		
	F1	Precision	Recall	F1	Precision	Recall	F1	Precision	Recall	F1	Precision	Recall
XGBoost	0.35	0.21	1.0	0.44	0.34	0.62	0.10	0.05	0.99	0.29	0.17	0.89
GPT-4o	0.78	0.64	1.0	0.81	0.68	0.99	0.56	0.39	0.98	0.49	0.34	0.88
SemPy (adapted)	0.41	0.26	1.0	0.43	0.50	0.38	0.29	0.22	0.47	0.26	0.29	0.23
Auto-BI (adapted)	0.56	0.39	1.0	0.52	0.61	0.45	0.32	0.42	0.26	0.08	0.15	0.05
Nexus-XGBoost (Ours)	**0.88**	0.78	1.0	0.72	0.58	0.95	0.57	0.44	0.85	0.50	0.42	0.63
Nexus-GPT-4o (Ours)	**0.88**	0.78	1.0	**0.89**	0.88	0.90	**0.83**	0.86	0.80	**0.61**	0.49	0.80



TPC-H **TPC-DS** **BIRD-SQL** **REAL**

Figure 6: F1 score comparison of base models, Nexus-Fast, and Nexus-EM across four datasets.

positives (evidenced by its lower precision relative to its nearly perfect recall). For the unseen REAL dataset, GPT-4o correctly predicts some true joins but suffers from a high false positive rate.

The pre-trained model serves as a modular component within Nexus. While we use GPT-4o to exemplify a strong prior, it can be replaced by any advanced ML model. The results of Nexus-XGBoost and Nexus-GPT-4o highlight the robustness of our framework: while a more accurate prior naturally yields better results, Nexus delivers adequate quality even when initialized with a significantly weaker model. In particular, Nexus-XGBoost matches the high result quality of Nexus-GPT-4o on TPC-H, even though its underlying XGBoost prior is substantially weaker than the GPT-4o prior (F1 scores of 0.35 vs. 0.78).

Auto-BI and SemPy underperform on all datasets. This outcome is not surprising, as both methods were originally designed assuming access to data values. The lack of data access prevents the use of inclusion dependency checks for pruning candidates, leading to higher false positive rates. Additionally, Auto-BI's performance relies on a reasonably effective base model, and its results are significantly compromised by the absence of features derived from data values. As shown in Section 4.7, both methods demonstrate much-improved results when granted full data access. However, for the specific task of inferring a join graph from metadata alone, neither Auto-BI nor SemPy is competitive.

Finally, we observe a consistent trend: as dataset complexity increases, from simple single-schema settings (TPC-H and TPC-DS) to multi-source scenarios (BIRD-SQL) and complex production environments (REAL), result quality declines across all approaches. Specifically, all methods suffer from high false positive rates (and consequently low precision) in multi-source datasets. Rather than painting a rosy picture based on synthetic data, the REAL dataset provides a true measure of what existing methods can achieve in actual production settings (datasets from multiple sources residing in data lakes or lakehouses).

4.3 Fast Mode Inference

By default, Nexus leverages LLMs within its EM algorithm to infer semantic entity types of columns and perform soft type matching. While powerful, these LLM API calls can incur increased latency and considerable monetary costs. For latency- or budget-sensitive scenarios, we offer Nexus-Fast, a fast mode of Nexus that exclusively employs low-rank matrix completion (no iterative EM algorithm) on the core submatrix (as discussed in Section 3.6).

For clarity in our comparison, we refer to our default algorithm with EM iterations as Nexus-EM to distinguish from Nexus-Fast. In Figure 6, we compare the F1 score of Nexus-Fast and Nexus-EM, using XGBoost and GPT-4o as base models for scoring join candidates. Overall, Nexus-Fast can significantly improve result quality of both base models across four datasets while Nexus-EM achieves the best results.

When a weak base model is used, like XGBoost, Nexus-EM empowered by LLM delivers stronger results than Nexus-Fast, although it has a longer runtime (as shown in the next section). Specifically, Nexus-EM outperforms Nexus-Fast by 38, 21, and 45 F1 points on the TPC-H, TPC-DS, and BIRD-SQL datasets, respectively. When the weak model provides poor priors, as seen with BIRD-SQL, Nexus-Fast barely improves result quality by 2 F1 points. Nevertheless, when the same base model has some predictive value as with TPC-H, TPC-DS, and REAL, Nexus-Fast achieves a boost of 15, 7 and 19 F1 points, respectively.

In contrast, when a strong base model like GPT-4o is employed, Nexus-Fast yields result quality comparable to Nexus-EM. The

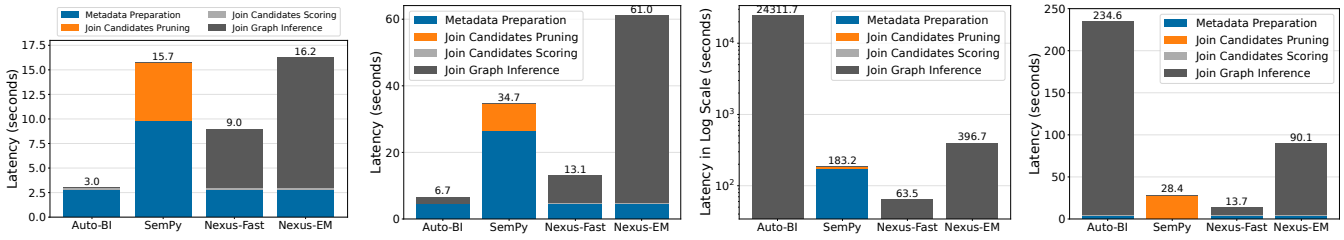


Figure 7: Efficiency comparison on TPC-H, TPC-DS, BIRD-SQL, and REAL datasets. Auto-BI shares Nexus's implementations of metadata preparation, join candidates pruning and join candidates scoring.

quality gap significantly shrinks, narrowing to only 1 F1 point on TPC-DS, 4 on BIRD-SQL, 3 on REAL, and no difference on TPC-H. Compared to the base model alone, Nexus-Fast yields improvements of 10, 7, 23, and 9 F1 points on these four datasets. The next section will elaborate on the latency advantages of Nexus-Fast.

4.4 Efficiency Comparison

Figure 7 shows the efficiency comparison of the four methods across all datasets. We report the runtime of Nexus-Fast and Nexus-EM using the XGBoost base model here.

We decompose end-to-end latency into four key components: metadata preparation, join candidates pruning, join candidates scoring, and join graph inference, as illustrated in our pipeline (Figure 5). Auto-BI, Nexus-Fast, and Nexus-EM incorporate all four components, whereas SemPy only involves metadata preparation and join candidates pruning. To ensure a fair comparison in our metadata-only adaptation, we replaced the first three components of Auto-BI with the more efficient implementations from Nexus.

Among all methods, Nexus-Fast demonstrates competitive efficiency, particularly on multi-source datasets with large numbers of tables and columns. On the BIRD-SQL and REAL datasets, Nexus-Fast is more than 6x faster than Nexus-EM. This performance gap is expected, as Nexus-Fast exclusively performs low-rank matrix completion on the core submatrix, whereas Nexus-EM incurs the additional cost of LLM API calls for semantic column type checks in each iteration. The latency for Nexus-EM also depends on the number of iterations we run the algorithm for. Here, we run Nexus-EM for five iterations, which is sufficient to achieve the best result on each dataset. Nevertheless, Nexus-EM completes join detection within a few minutes even on large multi-source datasets. Auto-BI exhibits the highest latency on BIRD-SQL and REAL datasets, running 382x and 17x slower than Nexus-Fast, respectively. This suggests that Auto-BI's graph-based optimization technique struggles to scale to larger multi-source datasets.

Together, Figure 6 and Figure 7 demonstrate that Nexus-Fast offers an attractive trade-off between quality and efficiency when utilizing a reasonably accurate base model.

4.5 Ablation Study

We conduct an ablation study to analyze the impact of join candidates pruning and key hyperparameters on the F1 score.

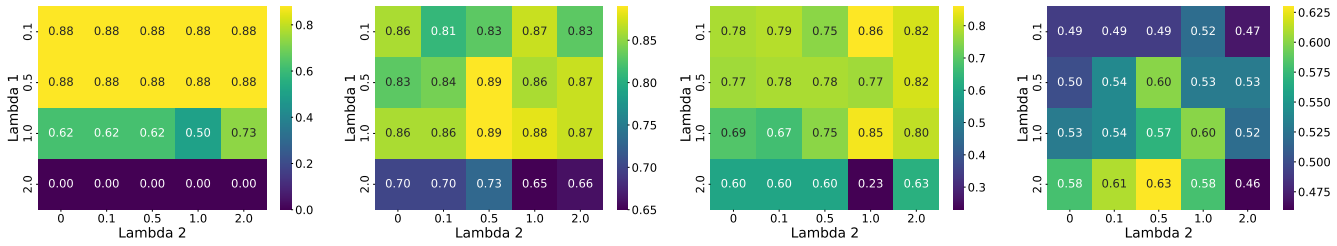
Table 7: Join candidates pruning using only metadata.

	TPC-H	TPC-DS	BIRD-SQL	REAL
# Join Candidates	1,830	90,100	324,415	667,590
# Pruned	1647	86,749	308,789	666,175
Pruning Rate	90.0%	96.3%	95.2%	99.8%
# False Negatives	0	0	1	6

4.5.1 Join Candidates Pruning. Table 7 gives the statistics of our join candidates pruning strategy using only metadata. The strategy consistently achieves a pruning rate exceeding 90%, reaching a peak of 99.8% on the REAL dataset. This highlights the effectiveness of our pruning strategy and it is key to saving computational and monetary costs of our join graph inference including the invocation of LLMs. While pruning inherently trades result quality for efficiency, the impact on accuracy is negligible: very few true positives in the ground truth are removed. Those rare misses are primarily attributed to data noise where join pairs violate the PK/FK constraints by failing the min/max value range check.

4.5.2 Weights λ_1 and λ_2 in the LRMC objective function. The parameters λ_1 and λ_2 balance between low-rank regularization and sparsity regularization in our optimization objective of low-rank matrix completion (equation 5). We evaluate a grid of 20 combinations, comprising four values for λ_1 (0.1, 0.5, 1.0, 2.0) and five values for λ_2 (0, 0.1, 0.5, 1.0, 2.0). Figure 8 presents the F1 score heat maps for Nexus-GPT-4o. Overall, Nexus-GPT-4o yields optimal results with moderate regularization strength. Specifically, $\lambda_1 \in \{0.5, 1.0\}$ performs best on TPC-H, TPC-DS, and BIRD-SQL, whereas a larger λ_1 is optimal for REAL. This aligns with the dataset properties: REAL has the lowest normalized rank (0.02 vs. 0.20 for TPC-H, as shown in Table 5), necessitating stronger low-rank regularization. Due to space constraints, we omit heatmaps for Nexus-XGBoost where it exhibits a similar pattern.

4.5.3 Thresholds in the EM algorithm. Our EM algorithm introduces two parameters: *low_threshold* and *high_threshold*. Since we observe base models exhibit a low false negative rate, we fix *low_threshold* to 0.5, which is also the decision threshold of base models. In other words, if a base model predicts a candidate pair negative, we bypass the expensive LLM semantic check for this pair in the EM algorithm. This allows Nexus-EM to focus on pairs predicted as positive, ensuring execution completes within minutes, even for larger datasets. On the other hand, *high_threshold*



TPC-H **TPC-DS** **BIRD-SQL** **REAL**

Figure 8: F1 score heat maps of Nexus-GPT-4o w.r.t loss weights λ_1 and λ_2 in the low-rank matrix completion objective function.

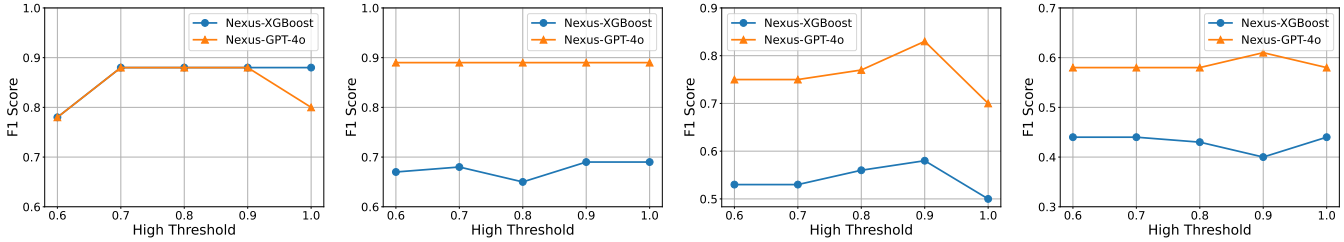


Figure 9: F1 score w.r.t high_threshold in the Nexus-EM algorithm across TPC-H, TPC-DS, BIRD-SQL, and REAL datasets.

controls how soon we mark a type-compatible candidate pair as a definitive positive. Figure 9 illustrates the F1 score sensitivity to *high_threshold* for Nexus-XGBoost/GPT-4o. Overall, both methods are relatively robust to the change of *high_threshold*.

4.6 When Query Logs are Available

In practice, query logs are commonly available and can provide partial, if not complete, set of true join relationships. We evaluate Nexus's result quality under varying percentages of available joins, which are randomly sampled from the ground truth. Figure 10 shows the F1 score of Nexus-EM as the proportion of known joins increases across the four datasets.

Overall, increased availability of true joins via query logs leads to improved result quality for Nexus. This is because sampled true joins provide valuable structural signals regarding the connectivity of the underlying ground truth adjacency matrix. For instance, on TPC-DS with 40% of true joins available, Nexus-GPT-4o achieves an F1 score of 0.91 and Nexus-XGBoost reaches an F1 score of 0.78. When the availability of true joins increases to 80%, these scores rise to 0.96 and 0.92, respectively. The benefits of known joins are also significant on the REAL dataset, where F1 scores increase by over 20 points for both Nexus-XGBoost and Nexus-GPT-4o when 80% of true joins are known. In contrast, the base models, XGBoost and GPT-4o, exhibit negligible improvement from increased known joins, as indicated by the flat dashed lines in Figure 10. This is because base models already have (nearly) perfect recall on all datasets (see Table 6). This gives strong evidence that the observed quality gains are driven specifically by our EM algorithm based on low-rank matrix completion.

4.7 When Data Values are Available

While Nexus is designed for enterprise settings where data access is restricted, it remains highly effective when data values are available. We compare Nexus-EM against Aurum, DeepJoin, XGBoost, Auto-BI, and SemPy on the TPC-H and TPC-DS datasets. The multi-source datasets are excluded in the interest of space. With full data access, we switch to Auto-BI's XGBoost model trained on additional features from data values across all relevant methods, replacing the metadata-only models in previous experiments.

Figure 11 shows F1 scores for all six methods. Unlike the metadata-only setting, XGBoost, Auto-BI, and SemPy now perform competitively, highlighting their heavy reliance on data values. This contrast reinforces our earlier observation: when data signals are absent, utilizing the global join graph structure is critical for inferring joinability. Nevertheless, Nexus remains the top performer on both TPC-H and TPC-DS. Even though these datasets conform to the strict snowflake schema that Auto-BI is optimized for, Nexus still outperforms Auto-BI on TPC-DS.

In addition to better ML predictions using value-based features, access to data values enables inclusion dependency tests, an effective technique for pruning join candidates. This test effectively eliminates many false positives that might otherwise appear highly joinable based on metadata alone. On TPC-DS, the number of join candidates drops from 3351 to 1232. On TPC-H, the reduction is even more dramatic, from 183 candidates to just 30. This aggressive pruning is a key reason why XGBoost, Auto-BI, and Nexus all achieve perfect F1 scores on TPC-H.

Aurum and DeepJoin, representing approaches for the "joinable table search" problem, trail behind all other methods. They require users to specify query columns and focused on a subset of true join keys in their original evaluation. In the setup of inferring full join graphs, both approaches produce a significant number of false positives when using all columns with unique values as query columns. DeepJoin further requires setting a fixed k for top-k search and is originally designed for string-typed columns, while over 80% of joins in the test datasets involve numerical columns.

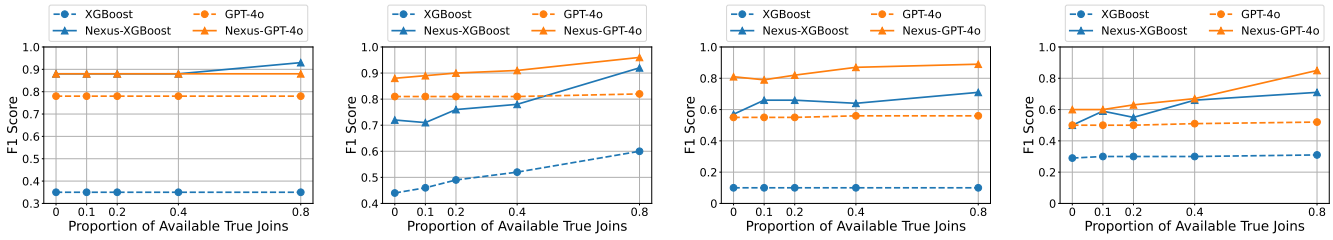


Figure 10: F1 score of base models (dashed lines) and Nexus-EM (solid lines) w.r.t the proportion of available true joins.

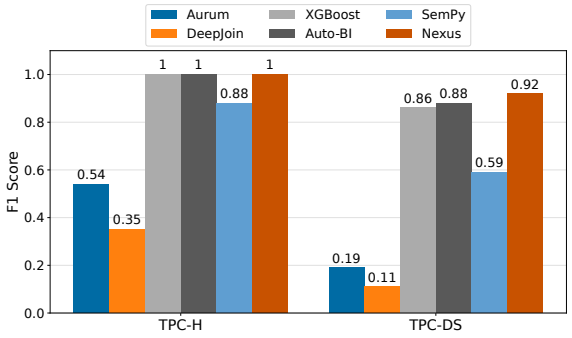


Figure 11: Comparison of F1 score on TPC-H and TPC-DS when data values are available.

5 RELATED WORK

Join detection has been studied extensively in the literature, in the context of PK/FK detection in relational databases [6, 18, 29, 32], BI model construction for analytical workloads [1, 23], and related table search in Web corpora [3, 4, 28, 31] and data lakes [2, 11, 13–16, 25, 33–35]. Prior work typically targets one of two scenarios: (1) joinable table search, a "point query" setup that finds tables joinable with a given input table/column, and (2) join graph inference, which finds all join relationships within a collection of tables. This work focuses on the latter without requiring user inputs.

Traditional methods on join detection rely on heuristics like name similarity and value overlap [6, 26]. For example, Chen et al. [6] combine signals from table and column names with data values to distinguish true relationships from spurious candidates, using containment and other constraints to prune unlikely join candidates. Similarly, Jiang et al. [18] jointly detect primary and foreign keys using a comprehensive set of pruning rules. However, these methods are often computationally expensive and miss semantic matches. Zhang et al. [32] detect PK/FK relationships by measuring distributional similarity between columns with the Earth Mover's Distance, but this approach does not scale well, requiring over 2.5 hours to process 10 GB of data [6].

Embedding-based approaches represent table and column metadata as vectors and use approximate nearest neighbor (ANN) search to find join candidates [2, 8, 14]. While these methods can uncover non-obvious semantic joins, their similarity metrics do not guarantee correctness, often leading to a high rate of false positives.

Recent work has shifted toward ML-based methods. Rostin et al. [29] propose learning PK/FK relationships from known foreign keys using various ML models. However, subsequent work has shown that such pretrained models often fail to generalize to unseen datasets [6, 23]. In Nexus, we address this limitation, by adopting an EM algorithm that iteratively improve the predictions from the pretrained ML model with the guidance of LLMs.

Our work is most closely related to recent approaches by Bharadwaj et al. [1] and Lin et al. [23], which aim to automatically and efficiently construct complete join graphs with minimal user input. Similar to earlier work [29], the method from Bharadwaj et al. [1] relies on ML models trained on large-scale data. While effective on in-distribution datasets, their approach does not address out-of-distribution generalization and overlooks the global structure of the join graph. Auto-BI [23] target a specific BI setting, where it constructs a join graph by applying a k-Min-Cost-Arborescence optimization on top of ML predictions. However, its enforcement of a snowflake schema is overly restrictive for datasets of loose schema. In contrast, our approach is more general and makes no assumptions about workload characteristics. We base our method on the key observations that join graphs in real-world scenarios are highly sparse and exhibit a low-rank structure. By leveraging this insight, we demonstrate that our approach performs robustly across a diverse range of datasets.

Finally, most prior work, except [1], assumes access to data values, but value-based feature computation on large tables is computationally expensive and often impractical in enterprise settings due to compliance and audit requirements [1]. In contrast, Nexus is designed to operate solely on metadata, while still providing high-quality results in an efficient manner.

6 CONCLUSION

In this work, we introduce Nexus, a novel, metadata-only solution for join graph inference designed specifically for enterprise environments with stringent data privacy requirements. Our approach is founded on key insights derived from analyzing numerous real-world database schemas: join graphs are not only highly sparse but also consistently exhibit a low-rank structure. We exploit these properties by formulating join graph inference as a low-rank matrix completion problem. To overcome the poor generalization of existing pretrained models, we propose a novel EM algorithm that iteratively refines join probabilities by incorporating semantic compatibility assessments from LLMs. Extensive evaluations, including on a real production dataset, demonstrate that Nexus achieves significant improvements in both effectiveness and efficiency.

7 ARTIFACTS

Reproducibility: As Microsoft has filed a patent for this work, the code cannot be open-sourced. However, we provide all LLM prompts and hyperparameters in the supplemental materials[1] to ensure reproducibility. Notably, our approach was recently independently reproduced and validated by the preprint arXiv:2603.04176.

References

[1] Sagar Bharadwaj, Praveen Gupta, Ranjita Bhagwan, and Saikat Guha. 2021. Discovering Related Data At Scale. *Proc. VLDB Endow.* 14, 8 (2021), 1392–1400.

[2] Alex Bogatu, Alvaro A. A. Fernandes, Norman W. Paton, and Nikolaos Konstantinou. 2020. Dataset Discovery in Data Lakes. In *36th IEEE International Conference on Data Engineering, ICDE 2020, Dallas, TX, USA, April 20-24, 2020*. IEEE, 709–720.

[3] Michael J. Cafarella, Alon Y. Halevy, and Nodira Khoussainova. 2009. Data Integration for the Relational Web. *Proc. VLDB Endow.* 2, 1 (2009), 1090–1101.

[4] Michael J. Cafarella, Alon Y. Halevy, Daisy Zhe Wang, Eugene Wu, and Yang Zhang. 2008. WebTables: exploring the power of tables on the web. *Proc. VLDB Endow.* 1, 1 (2008), 538–549.

[5] Emmanuel J. Candès and Benjamin Recht. 2009. Exact Matrix Completion via Convex Optimization. *Found. Comput. Math.* 9, 6 (2009), 717–772.

[6] Zhimin Chen, Vivek R. Narasayya, and Surajit Chaudhuri. 2014. Fast Foreign-Key Detection in Microsoft SQL Server PowerPivot for Excel. *Proc. VLDB Endow.* 7, 13 (2014), 1417–1428.

[7] Alexander L. Chistov and Dima Grigoriev. 1984. Complexity of Quantifier Elimination in the Theory of Algebraically Closed Fields. In *Mathematical Foundations of Computer Science 1984, Praha, Czechoslovakia, September 3-7, 1984, Proceedings (Lecture Notes in Computer Science, Vol. 176)*, Michal Chytil and Václav Koubek (Eds.). Springer, 17–31.

[8] Tianji Cong, James Gale, Jason Frantz, H. V. Jagadish, and Çagatay Demiralp. 2023. WarpGate: A Semantic Join Discovery System for Cloud Data Warehouses. In *13th Conference on Innovative Data Systems Research, CIDR 2023, Amsterdam, The Netherlands, January 8-11, 2023*. www.cidrdb.org.

[9] Tianji Cong, Fatemeh Nargesian, Junjie Xing, and H. V. Jagadish. 2025. OpenForge: Probabilistic Metadata Integration. *Proc. VLDB Endow.* 18, 9 (2025), 2914–2927.

[10] Arthur P Dempster, Nan M Laird, and Donald B Rubin. 1977. Maximum likelihood from incomplete data via the EM algorithm. *Journal of the royal statistical society: series B (methodological)* 39, 1 (1977), 1–22.

[11] Yuhao Deng, Chengliang Chai, Lei Cao, Qin Yuan, Siyuan Chen, Yanrui Yu, Zhaoze Sun, Junyi Wang, Jiajun Li, Ziqi Cao, Kaisen Jin, Chi Zhang, Yuqing Jiang, Yuanfang Zhang, Yuping Wang, Ye Yuan, Guoren Wang, and Nan Tang. 2024. LakeBench: A Benchmark for Discovering Joinable and Unionable Tables in Data Lakes. *Proc. VLDB Endow.* 17, 8 (2024), 1925–1938.

[12] Till Döhmen, Radu Geacu, Madelon Hulsebos, and Sebastian Schelter. 2024. SchemaPile: A Large Collection of Relational Database Schemas. *Proc. ACM Manag. Data* 2, 3 (2024), 172.

[13] Yuyang Dong, Kunihiro Takeoka, Chuan Xiao, and Masafumi Oyamada. 2021. Efficient Joinable Table Discovery in Data Lakes: A High-Dimensional Similarity-Based Approach. In *37th IEEE International Conference on Data Engineering, ICDE 2021, Chania, Greece, April 19-22, 2021*. IEEE, 456–467.

[14] Yuyang Dong, Chuan Xiao, Takuma Nozawa, Masafumi Enomoto, and Masafumi Oyamada. 2023. DeepJoin: Joinable Table Discovery with Pre-trained Language Models. *Proc. VLDB Endow.* 16, 10 (2023), 2458–2470.

[15] Raul Castro Fernandez, Ziawasch Abedjan, Famien Koko, Gina Yuan, Samuel Madden, and Michael Stonebraker. 2018. Aurum: A Data Discovery System. In *34th IEEE International Conference on Data Engineering, ICDE 2018, Paris, France, April 16-19, 2018*. IEEE Computer Society, 1001–1012.

[16] Javier Flores, Sergi Nadal, and Oscar Romero. 2021. Towards Scalable Data Discovery. In *Proceedings of the 24th International Conference on Extending Database Technology, EDBT 2021, Nicosia, Cyprus, March 23 - 26, 2021*. OpenProceedings.org, 433–438.

[17] Prateek Jain, Praneeth Netrapalli, and Sujay Sanghavi. 2013. Low-rank matrix completion using alternating minimization. In *Symposium on Theory of Computing Conference, STOC'13, Palo Alto, CA, USA, June 1-4, 2013*, Dan Boneh, Tim Roughgarden, and Joan Feigenbaum (Eds.). ACM, 665–674.

[18] Lan Jiang and Felix Naumann. 2020. Holistic primary key and foreign key detection. *J. Intell. Inf. Syst.* 54, 3 (2020), 439–461.

[19] Youri Kaminsky, Eduardo H. M. Pena, and Felix Naumann. 2023. Discovering Similarity Inclusion Dependencies. *Proc. ACM Manag. Data* 1, 1 (2023), 75:1–75:24. doi:10.1145/3588929

[20] Yehuda Koren, Robert M. Bell, and Chris Volinsky. 2009. Matrix Factorization Techniques for Recommender Systems. *Computer* 42, 8 (2009), 30–37.

[21] Sebastian Kruse and Felix Naumann. 2018. Efficient Discovery of Approximate Dependencies. *Proc. VLDB Endow.* 11, 7 (2018), 759–772.

[22] Jinyang Li, Binyuan Hui, Ge Qu, Jiaxi Yang, Binhua Li, Bowen Li, Bailin Wang, Bowen Qin, Ruiying Geng, Nan Huo, Xuanhe Zhou, Chenhao Ma, Guoliang Li, Kevin Chen-Chuan Chang, Fei Huang, Reynold Cheng, and Yongbin Li. 2023. Can LLM Already Serve as A Database Interface? A BIg Bench for Large-Scale Database Grounded Text-to-SQLs. In *Advances in Neural Information Processing Systems 36: Annual Conference on Neural Information Processing Systems 2023, NeurIPS 2023, New Orleans, LA, USA, December 10 - 16, 2023*.

[23] Yiming Lin, Yeye He, and Surajit Chaudhuri. 2023. Auto-BI: Automatically Build BI-Models Leveraging Local Join Prediction and Global Schema Graph. *Proc. VLDB Endow.* 16, 10 (2023), 2578–2590.

[24] Matéo Mahaut, Laura Aina, Paula Czarnowska, Momchil Hardalov, Thomas Müller, and Lluís Màrquez. 2024. Factual Confidence of LLMs: on Reliability and Robustness of Current Estimators. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), ACL 2024, Bangkok, Thailand, August 11-16, 2024*. Association for Computational Linguistics, 4554–4570.

[25] Marc Maynou, Sergi Nadal, Raquel Panadero, Javier Flores, Oscar Romero, and Anna Queralt. 2024. FREYJA: Efficient Join Discovery in Data Lakes. *CoRR* abs/2412.06637 (2024).

[26] Microsoft. 2024. *SemPy Python Library*. https://learn.microsoft.com/en-us/fabric/data-science/semantic-link-power-bi?tabs=sql Accessed: 2025-4-24.

[27] Yudi Pawitan and Chris Holmes. 2024. Confidence in the Reasoning of Large Language Models. *CoRR* abs/2412.15296 (2024).

[28] Rakesh Pimplikar and Sunita Sarawagi. 2012. Answering Table Queries on the Web using Column Keywords. *Proc. VLDB Endow.* 5, 10 (2012), 908–919.

[29] Alexandra Rostin, Oliver Albrecht, Jana Bauckmann, Felix Naumann, and Ulf Leser. 2009. A Machine Learning Approach to Foreign Key Discovery. In *12th International Workshop on the Web and Databases, WebDB 2009, Providence, Rhode Island, USA, June 28, 2009*.

[30] Olivier Roy and Martin Vetterli. 2007. The effective rank: A measure of effective dimensionality. *2007 15th European Signal Processing Conference* (2007), 606–610.

[31] Anish Das Sarma, Lujun Fang, Nitin Gupta, Alon Y. Halevy, Hongrae Lee, Fei Wu, Reynold Xin, and Cong Yu. 2012. Finding related tables. In *Proceedings of the ACM SIGMOD International Conference on Management of Data, SIGMOD 2012, Scottsdale, AZ, USA, May 20-24, 2012*. ACM, 817–828.

[32] Meihui Zhang, Marios Hadjieleftheriou, Beng Chin Ooi, Cecilia M. Procopiuc, and Divesh Srivastava. 2010. On Multi-Column Foreign Key Discovery. *Proc. VLDB Endow.* 3, 1 (2010), 805–814.

[33] Yi Zhang and Zachary G. Ives. 2020. Finding Related Tables in Data Lakes for Interactive Data Science. In *Proceedings of the 2020 International Conference on Management of Data, SIGMOD Conference 2020, online conference [Portland, OR, USA], June 14-19, 2020*. ACM, 1951–1966.

[34] Erkang Zhu, Dong Deng, Fatemeh Nargesian, and Renée J. Miller. 2019. JOSIE: Overlap Set Similarity Search for Finding Joinable Tables in Data Lakes. In *Proceedings of the 2019 International Conference on Management of Data, SIGMOD Conference 2019, Amsterdam, The Netherlands, June 30 - July 5, 2019*. ACM, 847–864.

[35] Erkang Zhu, Fatemeh Nargesian, Ken Q. Pu, and Renée J. Miller. 2016. LSH Ensemble: Internet-Scale Domain Search. *Proc. VLDB Endow.* 9, 12 (2016), 1185–1196.

[1] https://github.com/superctj/nexus_supplemental_material